Yocto End-to-End: 20-Assignment Curriculum

Targets: QEMU (`qemux86-64`) for fast iteration, BeagleBone Black (`beaglebone-yocto`) for real hardware. **Host:** Ubuntu 24.04. **Rule of progression:** each assignment reuses artifacts, layers, and understanding from earlier ones. Don't skip. By #20 you assemble everything into one product build.

Legend for each task: **Goal** (what mastery it cements) · **Do** (concrete steps) · **Deliverable** (proof you succeeded) · **Trap** (the mistake that teaches the concept when you hit it).

TIER 1 — EASY (1–5): Get a build working, understand the machinery

Assignment 1 — First clean build + QEMU boot

- **Goal:** Prove the toolchain works; know what a "build" actually produces.
- **Do:** Clone `poky`, source the env, build `core-image-minimal` for `MACHINE=qemux86-64`, boot with `runqemu`. Log in.
- **Deliverable:** A booting shell in QEMU + the exact path of the kernel and rootfs images you booted.
- **Trap:** `runqemu` picking the wrong image/machine when multiple exist — teaches you that `MACHINE` and image name are separate axes.

Assignment 2 — Anatomy of the build directory

- **Goal:** Map the filesystem layout so nothing downstream is a black box.
- **Do:** Without building anything new, produce a written map of `tmp/`, `tmp/deploy/`, `tmp/work/`, `sstate-cache/`, `downloads/`, `conf/`. For one recipe (`busybox`), find its `WORKDIR`, its unpacked source, its `image/` (fakeroot install), and its packaged output.
- **Deliverable:** A one-page table: directory → what lives there → who writes it.
- **Trap:** Confusing `tmp/work/.../image/` (per-recipe staging) with `tmp/deploy/images/` (final deployable artifacts).

Assignment 3 — BitBake as a task engine

- **Goal:** See that BitBake runs a *task graph*, not "a build."
- **Do:** For `busybox`: run `bitbake -c listtasks busybox`, dump the environment with `bitbake -e busybox`, generate the dependency graph with `bitbake -g core-image-minimal`. Run a single task (`-c compile`), then force a clean rebuild of just that recipe (`-c cleansstate` then rebuild) and watch sstate hits/misses.
- **Deliverable:** The ordered task list for one recipe, and an explanation of what `-e` output tells you that the recipe file alone does not.
- **Trap:** Editing a recipe and seeing no change — because sstate served a cached result. This is the lesson.

Assignment 4 — Read three recipes like an engineer

- **Goal:** Fluency reading metadata before you write any.
- **Do:** Pick a library recipe, an application recipe, and one that inherits a class (e.g. `autotools` or `cmake`). For each, identify: `SRC_URI`, `SRCREV` / `PV`, `LICENSE` / `LIC_FILES_CHKSUM`, `DEPENDS` vs `RDEPENDS`, and which `do_*` tasks come from the recipe vs the inherited class.
- **Deliverable:** For each recipe, one sentence: "what would break if I removed `DEPENDS`?" vs "…`RDEPENDS`?"
- **Trap:** Assuming `DEPENDS` and `RDEPENDS` are the same thing — build-time vs run-time is a distinction you must internalize now.

Assignment 5 — Bend the image with `local.conf`

- **Goal:** Change build output without writing a recipe.
- **Do:** In `local.conf`: add packages via `IMAGE_INSTALL:append`, enable `EXTRA_IMAGE_FEATURES = "debug-tweaks ssh-server-dropbear"`, switch package format (`PACKAGE_CLASSES = "package_ipk"`), and add/remove a `DISTRO_FEATURES`. Rebuild `core-image-minimal`, boot, and verify each change on target (package present, ssh reachable, `.ipk` files in deploy).
- **Deliverable:** Before/after: the change made and the observable evidence on the running image.
- **Trap:** Adding a package that pulls a `DISTRO_FEATURE` you stripped → the recipe silently doesn't install what you expected. Teaches feature gating.

TIER 2 — INTERMEDIATE (6–10): Author your own metadata

Assignment 6 — Your own layer

- **Goal:** Own a clean, correctly-registered layer — the container for everything you build from here on.
- **Do:** `bitbake-layers create-layer meta-custom`, add it with `bitbake-layers add-layer`. Inspect the generated `conf/layer.conf`: understand `BBFILE_COLLECTIONS`, `BBFILE_PATTERN`, `BBFILE_PRIORITY`, and `LAYERSERIES_COMPAT`.
- **Deliverable:** `bitbake-layers show-layers` output with `meta-custom` present, and a one-line explanation of what `BBFILE_PRIORITY` decides.
- **Trap:** `LAYERSERIES_COMPAT` mismatch after a release bump — teaches that layers are pinned to release codenames.

Assignment 7 — Recipe from scratch (and one from upstream)

- **Goal:** Write both a hand-rolled recipe and a "fetch-and-configure real software" recipe.
- **Do:** (a) A `hello_1.0.bb` for a local C source file with hand-written `do_compile` / `do_install`, packaged and installed to `/usr/bin`. (b) A recipe fetching a small autotools/cmake project from a real `SRC_URI`, letting the inherited class drive compile/install.
- **Deliverable:** Both binaries running on target; explain why (b) needed almost no `do_*` overrides.
- **Trap:** Files not landing in the package (`QA` "installed-but-not-shipped" / empty package) — teaches `FILES:${PN}` and packaging split.

Assignment 8 — Custom image recipe + packagegroup

- **Goal:** Define *your* image as first-class metadata, not `local.conf` hacks.
- **Do:** Write `my-image.bb` inheriting `core-image`. Curate `IMAGE_INSTALL`. Create a `packagegroup-mystack.bb` and install the group instead of loose packages. Include your `hello` recipe from #7.
- **Deliverable:** `bitbake my-image` boots with your curated package set; show the packagegroup pulling its members.
- **Trap:** Duplicating package lists across images — the packagegroup is the DRY fix; feel the pain first.

Assignment 9 — Modify a recipe you don't own (`.bbappend`)

- **Goal:** Extend upstream recipes without forking them.
- **Do:** Write a `.bbappend` for an existing recipe (e.g. add a config file or a patch via `SRC_URI`, plus a `do_install:append`). Use `FILESEXTRAPATHS`. Confirm the append applied with `bitbake -e`.
- **Deliverable:** The upstream recipe's behavior changed on target, with the original recipe file untouched.
- **Trap:** Wrong `.bbappend` filename/version glob so it never applies silently — teaches the name-must-match rule.

Assignment 10 — `devtool` workflow

- **Goal:** Use the modern iterate-on-source workflow.
- **Do:** `devtool add` a new upstream package, `devtool modify` an existing one to patch its source in a git-tracked workspace, `devtool build`, then `devtool finish` to fold changes back into a layer as proper patches/recipes.
- **Deliverable:** A patch generated by `devtool finish` landing in `meta-custom`, plus an explanation of what the `workspace` layer is.
- **Trap:** Editing in `workspace/sources` and forgetting to `finish` → work lost on cleanup. Teaches where devtool's edits actually live.

TIER 3 — HARD (11–20): Hardware, BSP, kernel, distro, product

Assignment 11 — Build for real hardware (BeagleBone Black)

- **Goal:** Cross from emulation to silicon.
- **Do:** Add `meta-yocto-bsp` if needed, build `core-image-minimal` (or `my-image`) for `MACHINE=beaglebone-yocto`. Write the `.wic` to an SD card, boot the BBB, attach a serial console (115200 8N1), log in.
- **Deliverable:** Serial-console login on the physical board + the U-Boot boot log.
- **Trap:** Flashing the rootfs tarball instead of the `.wic`, or a partition/console mismatch → no console output. Teaches the wic vs tar vs raw distinction.

Assignment 12 — Kernel configuration the Yocto way

- **Goal:** Change kernel config reproducibly, not by hand-editing `.config`.
- **Do:** `bitbake -c menuconfig virtual/kernel`, capture the delta as a **config fragment** (`bitbake -c diffconfig`), and deliver it via a `linux-yocto` `.bbappend` using `SRC_URI += "file://my.cfg"` / `KERNEL_FEATURES`. Verify the option in the booted kernel (`/proc/config.gz` or `zcat`, or `dmesg`).
- **Deliverable:** A tracked `.cfg` fragment in `meta-custom` that provably flips one kernel option on the running BBB.
- **Trap:** Editing `.config` in `WORKDIR` and losing it on clean — the whole point of fragments.

Assignment 13 — Out-of-tree kernel module recipe

- **Goal:** Ship your own driver as a package.
- **Do:** Write a minimal char driver (or `hello` LKM), package it with a recipe inheriting `module`, install it, then `modprobe` / `insmod` on the BBB. Confirm with `lsmod` / `dmesg`. (Reuses your driver background from BBB/Zynq work.)
- **Deliverable:** Your module loading on target and printing to the kernel log.
- **Trap:** Kernel-version / `KERNEL_MODULE_AUTOLOAD` mismatch so the module builds but never loads at boot. Teaches autoload vs manual.

Assignment 14 — Device tree change on the BBB

- **Goal:** Enable hardware via DT, the core of board customization.
- **Do:** Enable a peripheral (I2C/SPI device, or a GPIO/cape) by patching the `.dts` or adding a DT overlay for `beaglebone-yocto`, delivered through your layer. Verify the device node appears (`/sys`, `/dev`, or a driver probe in `dmesg`).
- **Deliverable:** A new working device node on the BBB traceable to your DT change.
- **Trap:** Editing the DT in a place the build doesn't consume (wrong `KERNEL_DEVICETREE`/overlay path) — teaches how the machine selects DTBs.

Assignment 15 — Custom BSP layer + custom MACHINE

- **Goal:** Own the board definition instead of borrowing `beaglebone-yocto`.
- **Do:** Create `meta-mybsp`. Add `conf/machine/myboard.conf` deriving from the BBB machine: set `KERNEL_DEVICETREE`, `SERIAL_CONSOLES`, `MACHINE_FEATURES`, `IMAGE_FSTYPES`, `PREFERRED_PROVIDER_virtual/kernel`. Build your image for `MACHINE=myboard` and boot it on the BBB.
- **Deliverable:** A boot on real hardware where `MACHINE=myboard`, not `beaglebone-yocto`.
- **Trap:** Under-specifying the machine conf so it can't build a bootable image — teaches which machine variables are load-bearing.

Assignment 16 — Custom/pinned kernel recipe

- **Goal:** Control exactly which kernel you ship.
- **Do:** Pin a specific kernel: either a `linux-yocto` `.bbappend` pinning `LINUX_VERSION` / `SRCREV` with your defconfig, **or** a standalone kernel recipe fetching from a git `SRC_URI` with `KBUILD_DEFCONFIG`. Wire it via `PREFERRED_PROVIDER_virtual/kernel` in your machine conf.
- **Deliverable:** `uname -r` on the BBB matching the version you pinned, built from your recipe.
- **Trap:** `kernel-yocto` metadata vs a plain kernel recipe confusion — you'll learn which gives you the config-fragment machinery and which doesn't.

Assignment 17 — New board bring-up (near-scratch)

- **Goal:** Do the thing a BSP engineer is hired for.
- **Do:** Treat a variant board as "new": define its bootloader (a `u-boot` recipe/ `.bbappend` with the right `UBOOT_MACHINE` /defconfig), its machine conf, its kernel + DT, and a bootable image layout — assembled from near-zero in `meta-mybsp`. Emulate first where possible, then boot the BBB standing in for the new board.
- **Deliverable:** A documented bring-up: bootloader → kernel → DT → rootfs, each piece owned by your layer.
- **Trap:** Getting a kernel but no working U-Boot environment / boot script → board hangs pre-kernel. Teaches the full boot chain, not just Linux.

Assignment 18 — Custom image partitioning with `.wks`

- **Goal:** Control the on-disk image, not just its contents.
- **Do:** Write a custom `.wks` kickstart: bootloader partition + rootfs + a separate persistent data partition. Point `WKS_FILE` at it, build a `.wic`, flash, boot, and confirm the extra partition mounts on the BBB.
- **Deliverable:** `lsblk` / `mount` on target showing your partition scheme.
- **Trap:** Off-by-one on partition types/ `--source` plugins so U-Boot can't find the kernel — teaches how wic plugins map to boot requirements.

Assignment 19 — Custom distro layer

- **Goal:** Set *policy* across all builds — the top of the config hierarchy.
- **Do:** Create `meta-mydistro` with `conf/distro/mydistro.conf`: choose `DISTRO_FEATURES`, init manager (systemd vs sysvinit), `PACKAGE_CLASSES`, `PREFERRED_VERSION` pins, and distro identity vars. Build `my-image` with `DISTRO=mydistro` for the BBB.
- **Deliverable:** A boot where distro policy (e.g. init system) provably differs from poky's defaults, set entirely by your distro layer.
- **Trap:** Putting machine settings in the distro conf (or vice versa) — this assignment forces the distro/machine separation to click.

Assignment 20 — Capstone: the full product

- **Goal:** Integrate everything into one reproducible, shippable build.
- **Do:** Assemble a single build for the BBB using **all** your layers: `DISTRO=mydistro` + `MACHINE=myboard` (`meta-mybsp`) + `my-image` (`meta-custom`) including your app (#7), packagegroup (#8), kernel module (#13), kernel config fragment (#12), and DT change (#14). Then: generate a cross-SDK with `bitbake -c populate_sdk my-image` (and try the eSDK), stand up a **package feed** (ipk/rpm over HTTP) so the target can `install` / `upgrade` packages post-deploy, and pin a shared `sstate` /downloads mirror so the build is reproducible.
- **Deliverable:** One command builds the whole product from clean; a second developer could install the SDK, build your app against it, push it to the running BBB via the feed, and see it update. Write a short README documenting the layer stack and build steps.
- **Trap:** Layer priority / `.bbappend` collisions surfacing only when all layers coexist — the capstone exists to expose these integration seams.

How the tree fits together (map, not territory)

- **1–5** teach the *engine and layout* (BitBake, tmp/deploy, recipes, local.conf).
- **6–10** teach *authoring metadata* (layers, recipes, images, appends, devtool).
- **11–14** teach *touching hardware* (BBB boot, kernel config, modules, device tree).
- **15–18** teach *owning the board* (BSP layer, custom machine, custom kernel, bring-up, wic).
- **19–20** teach *owning policy and shipping* (distro layer, SDK, feeds, reproducibility).

The 20% that yields 80%: assignments **3, 5, 6, 9, 12, 15, 19**. If you internalize the task engine, config precedence, layer mechanics, appends, kernel fragments, machine config, and distro policy, everything else is application of those seven.

Anti-pattern checklist (inversion — guaranteed ways to stall)

- Editing files in `tmp/work` and expecting persistence (#3, #12).
- Confusing build-time `DEPENDS` with run-time `RDEPENDS` (#4).

- Forking upstream recipes instead of using `.bbappend` (#9).
- Hand-editing `.config` instead of config fragments (#12).
- Mixing distro policy and machine definition in one conf (#15, #19).
- Assuming a booting kernel means a booting board — U-Boot is part of the chain (#17).

Maintenance note (entropy)

Yocto knowledge is largely *learn-the-mechanics-once*, but the metadata churns per release. Redo the version-sensitive steps (LAYERSERIES_COMPAT, kernel recipe pins, machine variable names) whenever you jump LTS releases — that's where decay bites.